Meilenstein 2a – Komponenten

N = Instanzen im Mini-Batch · d = Eingänge (784 MNIST, 2 XOR) · K = Klassen (10 MNIST, 2 XOR) · n_in / n_out = Neuronen der vorherigen / eigenen Schicht · T / V = Anzahl Trainings- / Validierungsinstanzen

Komponente	Aufgabe	Wichtigste Methoden: Eingabe → Ausgabe (Formen)	Speichert zwischen zwei Aufrufen
Framework Klasse	Liest die Konfiguration, baut alle Komponenten über Fabrik-Methoden und steuert das Training über Epochen und Mini-Batches.	• `readEnvFile() → Config` • `createDataPrep(dataSetType) → DataPrep` • `validateConfig(config, dataPrep)` • `createLayers(topology, activations, calculation, seed) → inputLayer, hiddenLayers, outputLayer` • `createOptimizer(optimizerType) → Optimizer` • `train() → epochErrors (Länge epochCount)` • `trainStep(imageBatch N × d, labelBatch N × K) → errorValue (Zahl)`	• `config` • `dataPrep` • `mlp` • `optimizer` • `epochErrors`
Config dataclass	Hält alle Einstellungen aus der .env an einer Stelle.	• `keine`	• `dataSetType, topology, activations` • `calculationType, optimizerType` • `epochCount, batchSize, seed` • `learningRate, alpha (optional)`
DataPrep Klasse	Bereitet die Rohdaten einmal auf (umformen, skalieren, One-Hot, aufteilen) und liefert pro Epoche gemischte Mini-Batches.	• `__init__(rawImages, rawLabels, classCount, scaleFactor, validationSize, seed)` • `getMiniBatches(batchSize) → Liste von (imageBatch N × d, labelBatch N × K)` • `getValidationData() → validationImages V × d, validationLabels V × K`	• `trainImages T × d, trainLabels T × K` • `validationImages V × d, validationLabels V × K` • `rng`
MLP Klasse	Hält die Schichten und reicht Werte vorwärts und rückwärts durch alle Schichten.	• `forwardPropagation(currentBatch N × d) → prediction N × K` • `calculateLoss(prediction N × K, targetValue N × K) → errorValue (Zahl)` • `backwardPropagation(targetValue N × K) → Liste gradients, je (n_in+1) × n_out` • `getWeights() → Liste, je (n_in+1) × n_out` • `setWeights(Liste, je (n_in+1) × n_out)`	• `inputLayer` • `hiddenLayers` • `outputLayer`
Layer Interface	Gemeinsame Schnittstelle aller Schichten für den Vorwärtsschritt.	• `forwardStep(neuronValuesPrev N × n_in) → neuronValues N × n_out`	• –
InputLayer Klasse	Gibt die Eingabe unverändert weiter und prüft ihre Form.	• `forwardStep(N × d) → N × d`	• `neuronCount`
DenseLayer abstrakt	Gemeinsamer Teil aller rechnenden Schichten: −1 anhängen, net berechnen, Gradienten berechnen, Fehler an die vorherige Schicht weitergeben.	• `forwardStep(neuronValuesPrev N × n_in) → neuronValues N × n_out` • `backwardStep(backpropError N × n_out) → gradient (n_in+1) × n_out, deltaPrevLayer N × n_in` • `getWeights() → (n_in+1) × n_out` • `setWeights((n_in+1) × n_out)`	• `weights (n_in+1) × n_out, θ in Zeile 0` • `neuronValuesExt N × (n_in+1)` • `calculation, neuronCount, neuronCountPrev`
HiddenLayer Klasse	Wendet die wählbare Aktivierungsfunktion an und bildet δ = Schuld · f′(net).	• `activate(net N × n_out) → neuronValues N × n_out` • `calculateDelta(backpropError N × n_out) → delta N × n_out`	• `activationFunc` • `net N × n_out`

Komponente	Aufgabe	Wichtigste Methoden: Eingabe → Ausgabe (Formen)	Speichert zwischen zwei Aufrufen
OutputLayer Klasse	Wendet Softmax an, berechnet den Fehler (CCE) und bildet δ = o − t.	• `activate(net N × K) → prediction N × K` • `softmax(net N × K) → prediction N × K` • `categoricalCrossEntropy(prediction N × K, targetValue N × K) → errorValue (Zahl)` • `calculateDelta(targetValue N × K) → delta N × K`	• `prediction N × K`
Calculation Loop, Matrix Interface	Austauschbarer Rechenkern einer Schicht, mit Schleifen oder mit Matrizen (A4).	• `calculateNet(extendedInput N × (n_in+1), weights (n_in+1) × n_out) → net N × n_out` • `calculateGradient(extendedInput N × (n_in+1), delta N × n_out) → gradient (n_in+1) × n_out` • `calculateBackpropError(delta N × n_out, weights ohne Zeile 0 n_in × n_out) → deltaPrevLayer N × n_in`	• –
ActivationFunction ReLU, Sigmoid, Identity Interface	Austauschbare Aktivierungsfunktion der Hidden Layer (A1).	• `activationFunc(net N × n_out) → N × n_out` • `activationFuncDerivative(net N × n_out) → N × n_out`	• –
Optimizer Interface	Gemeinsame Schnittstelle der Trainingsverfahren (A2).	• `optimize(gradients Liste, weights Liste) → neue weights Liste, je (n_in+1) × n_out`	• –
GradientDescent Klasse	Gradientenabstieg mit fester Lernrate.	• `wie Optimizer`	• `learningRate`
Rprop Klasse	Rprop⁻ mit eigener Schrittweite pro Gewicht.	• `wie Optimizer`	• `n_pos, n_neg, delta_null, deltaMin, deltaMax` • `Listen delta und g_old, je (n_in+1) × n_out`
Adam Klasse	Adam mit gleitenden Mittelwerten pro Gewicht.	• `wie Optimizer`	• `alpha, beta_1, beta_2, epsilon` • `Listen m und v, je (n_in+1) × n_out` • `Schrittzähler t`